

C N E T 1 0 1 — C O M P U T E R N E T W O R K S — S P R I N G 2 0 2 6

GIU-Net Project

Student Approach Guide

How to read the project, how to plan four weeks, how to research what you don't know, and how to debug what doesn't work — without anyone telling you the answer.

Prepared by the Teaching Team

Companion to the Project Description & Expected Output decks

How to use this guide

THIS GUIDE IS

A map. It tells you which concepts the project rests on and where to look them up.

A planner. It suggests a four-week order so you don't write code before you understand UDP, or wire routers before you compute subnets.

A debugging mindset. It tells you what symptoms to expect and what tools to reach for first.

THIS GUIDE IS NOT

A solution. Every formula, derivation, and configuration in your submission must come from your team's own work.

A substitute for the spec. Read the Project Description in full. This deck only helps you navigate it.

An excuse to skip the textbook. The course material covers everything the project tests — open it first.

Why the project has two halves

You are building a small piece of software AND the network it would live on. The two halves are deliberately connected — Part 2 is the infrastructure for Part 1.

PART 1

Application Layer

A chat program that runs over raw UDP. You invent the rules: how a session begins, how messages are confirmed, when to retry, when to give up.

runs on

PART 2

Network Layer

A simulated campus network in Packet Tracer. This is the infrastructure that would carry your chat traffic between buildings, addressed and routed correctly.

TAKEAWAY Concepts in Part 1 (ports, addresses, reliability) only make sense once you've thought about Part 2 (subnets, routing, where packets actually go).

Read it like an engineer, not like a novel

Before you write a line of code, go through the Project Description with a highlighter. Pull out four kinds of things:

01

Constants

Values that depend on your team's IDs. Compute them once, write them at the top of every file, and check them in every screenshot.

02

Behaviours

Every sentence that says 'must' or 'should' is a test case. Make a checklist — handshake, ACK, retry, timeout, exit — and tick each one as you implement it.

03

Exact strings

Payload formats are checked byte-for-byte. Note every quoted string in the spec and reproduce it exactly: hyphens, capitalisation, punctuation, prefixes.

04

Deliverables

List every file the ZIP must contain before you start. If a file requires running on two machines, plan that session early — don't leave it for the last night.

Compute your team-specific values first

Three values from the spec depend on your specific team and appear in almost every deliverable. Compute them by hand, verify across team members, then hardcode them at the top of both source files.

Port number

WHERE TO FIND

Section 4.6 of the spec

WHY IT MATTERS

Used in your server, your client, and as the Wireshark filter.

Handshake strings

WHERE TO FIND

Section 4.4 of the spec

WHY IT MATTERS

The HELLO and OK payloads. Byte-for-byte comparison.

Base network

WHERE TO FIND

Section 5.3 of the spec

WHY IT MATTERS

Drives your entire VLSM table and every router config.

TIP Have every team member compute these independently and compare. A wrong digit here breaks every test for the rest of the month.

A four-week plan that actually works

W E E K 1	W E E K 2	W E E K 3	W E E K 4
Understand & Set Up ■ Form the team, register, agree on the Leader. ■ Compute the team-specific values together. ■ Install Java, Wireshark, Packet Tracer. Verify each runs. ■ Read the spec end-to-end as a group.	Part 1 – Build Incrementally ■ Start with a plain UDP echo on one machine. ■ Add the handshake. Test wrong strings too. ■ Add chat without ACKs. Then add ACKs. ■ Add timeout and retransmission last.	Part 2 – Design & Configure ■ Do all VLSM math on paper before opening Packet Tracer. ■ Build the topology. Verify each interface comes up. ■ Configure routing. Test pings hop by hop. ■ Add the web server and the custom page.	Integrate, Capture, Document ■ Live two-machine run. Capture Wireshark. Save the log. ■ Annotate every screenshot. Don't skip this. ■ Write the debugging story while it's still fresh. ■ Build the ZIP. Check every required file is inside.

Divide the work without splitting your knowledge

Four members, two halves — but everyone must understand every part. During evaluation, any member can be asked about any line of code.

M1	Protocol Lead	Drives Part 1: packet format, handshake, threading model. Pairs with M2 on testing.
M2	Reliability Lead	Owens ACK/retransmission logic and Wireshark capture session. Pairs with M1 on the live run.
M3	Network Lead	Drives Part 2: VLSM math, topology, routing. Pairs with M4 on verification.
M4	Documentation Lead	Webpage, ACL bonus, report writing, screenshot annotation. Pairs with M3 on diagrams.

WARNING Splitting work where one person handles all of Part 1 alone is the most common reason teams lose evaluation marks. Cross-pair on every component.

What you actually need to understand

These are the building blocks behind Part 1. Don't open your IDE until you can explain each one in your own words.

UDP vs TCP

Why UDP is connectionless and what you lose when you skip the TCP handshake. Why reliability has to be re-added by hand.

Sockets & datagrams

How DatagramSocket and DatagramPacket move bytes. The difference between the port a server listens on and the ephemeral port a client uses.

Byte order & encoding

Big-endian vs little-endian. Why network protocols agree on a byte order. How UTF-8 differs from raw bytes.

Threads & shared state

Why typing and receiving must run on separate threads. What 'volatile' means. How two threads coordinate without stepping on each other.

Timeouts & retries

How to bound a wait so a program doesn't hang forever. The difference between giving up and trying again.

Protocol design

How a payload format becomes a contract. Why exact strings matter. What 'parsing' a packet really means.

How to research what you don't know

Concrete starting points — none of which will hand you the project, all of which will teach you what's underneath it.

TEXTBOOK & COURSE

Kurose & Ross

Chapter 3 (Transport) for UDP fundamentals. Chapter 1 ideas about layering.

Course slides

Look back at lecture material on sockets and the transport layer. Your specific course covered this.

Lab notes

Earlier tutorial exercises on transport layer and network layer would significantly help on how things work behind the scene.

OFFICIAL DOCS

Oracle Java

DatagramSocket and DatagramPacket Javadoc. Read every method you call.

RFC 768

The UDP specification. Two pages. Read it.

Wireshark User Guide

Display filters, the Data field, and how to export captures as .pcapng.

SEARCH QUERIES

For Java basics

"java DatagramSocket receive example", "java big endian short bytes".

For concurrency

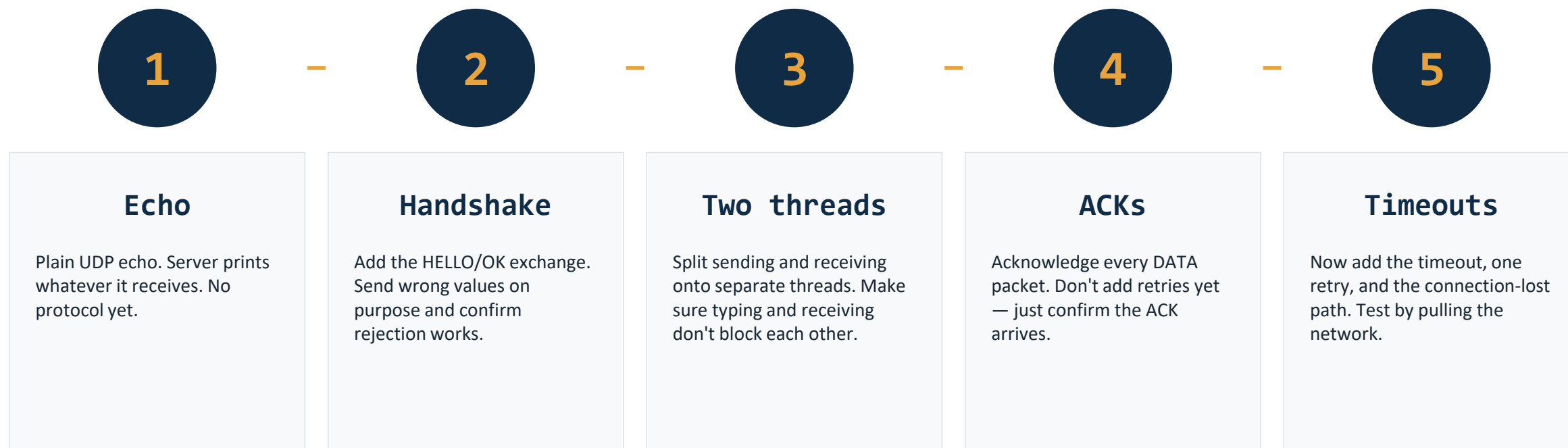
"java thread reading stdin while another receives", "volatile vs synchronized".

Typical search clause

"How to send and receive UDP packets"
"Hello World UDP Chat Program"

Build it in five small steps, not one big one

The fastest path to a working submission is the smallest first step. Each stage below should run, end-to-end, before you start the next.



Network concepts you need to ground

Open the textbook before you open Packet Tracer. Each concept below maps directly to a required task.

Binary & prefixes

Why /22 means 22 ones followed by 10 zeros. How a prefix length translates to a subnet mask and a host count.

VLSM

Why you allocate largest to smallest. How to find the smallest block that fits a given host count without wasting addresses.

Subnet anatomy

Network address, first usable, last usable, broadcast. How to compute each from a prefix.

Default gateways

Why every PC needs one. What happens if it's missing or wrong. The relationship to its local router interface.

Static routing

What 'ip route' tells a router. Why each router needs a route for every non-local subnet it can't see directly.

ACLs

How an access list filters traffic, why direction (in/out) matters, and why placement on the right interface is half the battle.

Where to study the networking side

Cisco Packet Tracer has its own learning resources — and your course material is the most direct path. Use them in this order:

FOUNDATIONS

Kurose & Ross

Chapter 4 (Network Layer) for addressing and forwarding. Chapter 5 for routing principles.

Subnetting practice

Solve five VLSM problems on paper before touching Packet Tracer. Drawing the binary helps.

Course tutorials

Your VLSM and routing tutorial worksheets are direct rehearsals.

CISCO RESOURCES

NetAcad

Free Packet Tracer courses ("Introduction to Networks") cover every CLI command this project needs.

IOS command reference

When a command doesn't behave, search Cisco's official IOS reference, not random forum posts.

Packet Tracer tutorials

The official 'Help → Tutorials' inside Packet Tracer demonstrates Simulation Mode step by step.

VERIFICATION HABITS

show ip route

On every router, after every change. If a subnet isn't there, no traffic to it will ever leave.

show ip interface brief

Half of all 'ping doesn't work' problems are interfaces that are administratively down.

ping & tracer

From inside Packet Tracer's PC Command Prompt. tracertracert exposes routing paths even when ping succeeds.

Debug like a network engineer

Network bugs hide because the failure point isn't where the symptom shows up. These are the habits that find them fast.

01

Trust the wire, not the code

If your program says it sent something, open Wireshark and confirm. The wire never lies; the print statement might.

02

Isolate the layer

Application not working? Test the network with ping first. Network not working? Test the wire with link lights. Don't debug everything at once.

03

Reproduce the smallest case

If a 5-message chat fails, can a 1-message exchange succeed? Strip away every feature until something works, then add back one at a time.

04

Log everything, then read it

Tag every line of output with what produced it. When something fails, the answer is almost always somewhere in the last 30 lines of output you ignored.

Symptoms to watch for

If you see any of these during development, stop and investigate — they signal a real problem you don't want to discover during evaluation. We're naming the symptoms; the fix is yours to find.

WHERE	SYMPTOM YOU MIGHT SEE	WHAT IT TYPICALLY SIGNALS
Part 1	Received messages contain trailing garbage or null characters.	You're reading more bytes from the buffer than the packet actually contained.
Part 1	Every message times out and retries, even though both programs run.	The ACK is being sent but not recognised — the strings on the two sides don't match exactly.
Part 1	Typing on one side freezes incoming messages from the other.	Sending and receiving are running on the same thread — concurrency is the wrong way around.
Part 2	Two PCs in the same subnet can ping each other; PCs in different subnets cannot.	A routing problem, not an addressing one. Check <code>`show ip route`</code> on each router.
Part 2	An interface won't come up even though the cable looks right.	Almost always 'no shutdown' missing — or the wrong cable type between two routers.

What graders are looking for

WRITING THE REPORT

Make every claim specific. "Chose 3 seconds because shorter values triggered false timeouts on the campus WiFi" beats "3 seconds seemed reasonable."

Show your VLSM working. Manual binary calculations earn marks; calculator output without working does not.

Tell a real debugging story. Symptom you saw, theory you had, tool you used, what was actually wrong. Generic descriptions are immediately recognisable.

Annotate every screenshot. Arrows, labels, captions. A raw screenshot with no annotation suggests you don't know what it shows.

LIVE EVALUATION

Practise the modification challenge. Before the slot, try changing your own timeout, keyword, or port. If you can do it in five minutes, you'll do fine.

Every member should be able to defend every part. The evaluator picks the line, not you. Walk through each other's code as a team the day before.

Bring your own running environment. Two laptops on the same network, Wireshark installed, Packet Tracer open. Don't troubleshoot tools during evaluation.

Be honest about what you don't know. "I'm not sure, but here's how I'd find out" earns more credit than a confident wrong answer.

One last thing.

This project is designed so that the work is the point.

The handshake string, the port number, the IP block, the webpage — all of them are tied to your team's specific IDs. AI assistants and online repositories don't have those numbers. They can produce code that looks right, but a submission that doesn't match your team's values is trivially detectable, and so is one where no member can explain what their code does.

We're not telling you not to use AI as a tutor — go ahead and ask one to explain UDP, or to clarify how threads work. **Just don't ask it to do the project for you.**

You'll lose more than marks — you'll lose the only chance you have to learn the material.

Good luck. We're rooting for you.